

Starter Project 3 Code Review

This assignment was to design and review a Python program to the Boggle word search problem. The program takes as input a two-dimensional grid of characters and a list of valid dictionary words, then returns all words that can be formed by traversing adjacent letters in the grid. Adjacency includes horizontal, vertical, and diagonal neighbors, and each cell may only be used once per word. To solve the problem, I implemented a depth-first search algorithm that explores all letter combinations from each cell in the grid. Because DFS alone can be time-consuming, I incorporated a prefix-based pruning strategy. This approach generates all valid prefixes from the dictionary and ensures that the search only continues when the current partial word matches a valid prefix. This significantly reduces unnecessary recursive calls and improves overall performance. The solution is in the Boggle class, which includes helper methods for checking input, generating prefixes, and doing a recursive traversal. This modular structure improves readability and allows each component of the program to serve a clear purpose.

While reviewing my partner's code, I noticed that their code repeated in ways that could have been avoided, such as calculating values inside loops instead of storing them. Also, their code did not properly handle edge cases like empty inputs or invalid data types, which could lead to runtime errors. Another concern was the lack of structure. Most of the code was contained within a single function, making the code difficult to read. I recommended breaking the code into smaller functions to improve clarity. Lastly, I pointed out that some assumptions were hardcoded, such as fixed grid dimensions, which limited the code's flexibility.

Based on the feedback I received, I made several improvements to my code. First, I changed the dictionary by converting all words to lowercase and storing them in a set. This made it so I don't need to constantly perform the lowercase conversion during the DFS process and improve performance. Second, I changed the way the visited set was handled. I updated the set in place by adding a cell before the recursion. This change reduced memory usage. I also improved variable naming throughout the code, replacing unclear names with more descriptive ones. I also made the valid function better by making it perform more thorough input validation. Finally, I reorganized parts of the code to better separate the concerns. For example, dictionary preprocessing is now clearly distinct from the DFS logic. These changes made the code cleaner and easier to understand.

After adding the improvements, I used static analysis tools such as pylint to evaluate the quality of my code. One common issue was inconsistent naming, so I updated all variable and function names to follow Python's snake_case standard. I also fixed line length violations by breaking long lines into shorter ones. In addition, I removed redundant computations and simplified complex expressions where possible. Some functions were refactored to reduce their complexity and improve readability. I also ensured that there were no unused variables or unnecessary imports. After making these changes, the code is closer to the standard Python style guidelines and got a higher quality rating from the analysis tools.

To check that my changes did not have any new errors, I used regression testing on the updated implementation. I ran tests on the original test cases and confirmed that the output was consistent. I then tested some edge cases, like empty grids, empty dictionaries, and words that don't meet the minimum length. These tests ensured that the program handles inputs correctly. Also, I created custom test cases with different grid combinations to evaluate the correctness of the solution. I also compared performance before and after the optimizations to confirm that the improvements were effective. All tests passed successfully, showing that the updated code maintains correctness while improving efficiency and reliability.

This assignment provided lots of insight into the importance of code reviews in professional software development. I learned that even when a program functions correctly, there are often opportunities to improve its efficiency, readability, and structure. Receiving feedback helped me identify weaknesses in my approach, particularly in terms of performance and unnecessary computations. It also showed me how small changes, such as better variable naming or avoiding repeated operations, can have a significant impact. Providing feedback to my partner was equally beneficial, as it required me to critically analyze another implementation and apply best practices. This process strengthened my understanding of algorithm design and code quality. Overall, this experience reinforced the idea that software development is a collaborative and iterative process. Code reviews, testing, and refinement are essential steps in producing high-quality, maintainable software.